Below is a full package you can give to the class: 20 projects, each with objectives, roles, resources, and timelines, plus report templates, a common rubric, and how GenAI use will be neutralized.

General Structure for All Projects

- **Teams**: 3 students
- **Duration**: 4 weeks
- **Deliverables (common)**
  - Midterm report (IEEE-style, ~4 pages) – end of Week 3
  - Final report (IEEE-style, ~6–8 pages) – end of Week 6
  - 20-minute presentation + 5–10 minutes Q&A
  - GitHub repository with:
    - Code
    - `README.md` (problem, method, data, how to run)
    - Reproducible experiments (scripts, seeds, configs)
- **Grading emphasis**: Reproducibility, understanding, and incremental originality (ablation, small extension, or strong analysis). Aim for **publishable quality**.

Hardware/software assumptions:

- CPU laptop with ≥8 GB RAM (no GPU assumed)
- Python 3.9+; PyTorch or TensorFlow + common libraries (NumPy, Pandas, scikit-learn, Matplotlib/Seaborn, Jupyter)

---

20 Capstone Projects

---

1. Lightweight Super-Resolution for Mobile Devices

Project Title: Efficient Image Super-Resolution with Lightweight CNNs

Project Aims & Objectives:

1. Reproduce a lightweight single-image super-resolution (SISR) model on a small dataset.
2. Compare performance vs. a classical baseline (e.g., bicubic) under CPU-only constraints.
3. Explore a minor extension (e.g., reduced model depth, pruning, or quantization).
4. Analyze trade-off between PSNR/SSIM and inference time on laptops.

Scope of Work:

- Implement and train a compact SISR network (e.g., based on IMDN or ESRGAN-lite) on low-resolution to high-resolution image pairs.
- Evaluate quantitatively (PSNR/SSIM) and qualitatively.
- Out of scope: large-scale training on full DIV2K.

Expected Deliverables:

1. Trained SISR model and scripts.
2. Comparative evaluation report and plots.
3. Final report and presentation.

Prerequisite Knowledge / Skills:

- Python, PyTorch/TensorFlow
- Basic CNNs, image processing (resizing, PSNR/SSIM)

Role of Each Student:

- **Student A (Model/Implementation Lead):** Implement model, training loops, and baseline.
- **Student B (Data & Evaluation Lead):** Preprocess data, compute metrics, visualization.
- **Student C (Analysis & Reporting Lead):** Hyperparameter tuning, ablations, write most of report/presentation.

Paper(s) Link:

- Hui et al., "Lightweight Image Super-Resolution with Information Multi-Distillation Network," ACM MM 2019. https://doi.org/10.1145/3343031.3350928
- Wang et al., "ESRGAN: Enhanced Super-Resolution Generative Adversarial Networks," ECCV Workshops 2018 (as a reference). https://arxiv.org/abs/1809.00219

Data Link:

- DIV2K (use a small subset, e.g., 100 images): https://data.vision.ee.ethz.ch/cvl/DIV2K/

Code Link (reference, not to blindly copy):

- IMDN PyTorch implementation: https://github.com/Zheng222/IMDN

Timeline (4 weeks):

- **Week 1:** Read paper; subset & preprocess DIV2K; implement baseline (bicubic).
- **Week 2:** Implement lightweight model; run first small-scale training; midterm report.
- **Week 3:** Tune hyperparameters; compare variants (e.g., shallower vs deeper); collect metrics.
- **Week 4:** Final experiments; write final report; prepare presentation; tidy GitHub.

Additional Notes:

Keep resolution small (e.g., ×2, small crops) to run training within hours on CPU.

---

2. ECG Arrhythmia Classification with Lightweight 1D CNNs

Project Title: Interpretable ECG Arrhythmia Classification on Resource-Constrained Devices

Project Aims & Objectives:

1. Reproduce a 1D CNN for ECG beat classification.
2. Implement simple interpretability tools (e.g., saliency, Grad-CAM for 1D).
3. Compare with a classical approach (e.g., feature-based + SVM).
4. Optimize for CPU inference (e.g., limit parameter count).

Scope of Work:

- Train a CNN for arrhythmia classification on a subset of MIT-BIH.
- Out of scope: advanced signal processing beyond simple filtering/normalization.

Expected Deliverables:

1. Trained model and scripts.
2. Interpretability visualizations.
3. Comparative analysis vs. classical baseline.

Prerequisite Knowledge / Skills:

- Python, PyTorch/TensorFlow
- Basic DSP (filtering), 1D CNNs

Role of Each Student:

- **Student A:** Data preprocessing & classical baseline.
- **Student B:** CNN model implementation and training.
- **Student C:** Explainability experiments and reporting.

Paper(s) Link:

- Hannun et al., “Cardiologist-Level Arrhythmia Detection and Classification in Ambulatory ECG Monitoring Using a Deep Neural Network,” Nat. Med., 2019. https://doi.org/10.1038/s41591-018-0268-3
- Yildirim, “A Novel Deep Learning Model for Automated ECG Diagnosis,” IEEE Access, 2018 (method reference). https://doi.org/10.1109/ACCESS.2018.2791887

Data Link:

- MIT-BIH Arrhythmia Database (PhysioNet): https://physionet.org/content/mitdb/1.0.0/

Code Link (reference):

- Example ECG classification in PyTorch: https://github.com/antonior92/DeepECG

Timeline:

- **Week 1:** Study papers; download MIT-BIH; implement beat segmentation.
- **Week 2:** Implement CNN; baseline SVM; midterm report.
- **Week 3:** Train and evaluate; implement saliency maps.
- **Week 4:** Final experiments, error analysis; finalize report & presentation.

---

3. Deepfake Image Detection with Compact CNNs

Project Title: Deepfake Face Image Detection with Lightweight Convolutional Models

Project Aims & Objectives:

1. Reproduce a CNN-based deepfake detection approach on images.
2. Use a lightweight architecture (e.g., MobileNetV2) suitable for CPU.
3. Compare performance across different manipulation types.
4. Perform a robustness test (compression, noise).

Scope of Work:

- Train on a small subset of a deepfake dataset.
- Out of scope: video-based temporal modeling.

Expected Deliverables:

1. Trained classifier and scripts.
2. Robustness evaluation report.
3. Final report and presentation.

Prerequisite Knowledge / Skills:

- CNNs, transfer learning
- Image preprocessing

Role of Each Student:

- **Student A:** Data selection and labeling pipeline.
- **Student B:** Model training (MobileNet/ResNet18) and tuning.
- **Student C:** Robustness tests and reporting.

Paper(s) Link:

- Qi et al., “DeepFake Detection by Analyzing Convolutional Traces,” IEEE T-IP, 2020. https://doi.org/10.1109/TIP.2020.3007003
- Rossler et al., “FaceForensics++: Learning to Detect Manipulated Facial Images,” ICCV 2019. https://arxiv.org/abs/1901.08971

Data Link:

- FaceForensics++ (use limited subset): https://github.com/ondyari/FaceForensics

Code Link (reference):

- FaceForensics++ baseline code: https://github.com/ondyari/FaceForensics

Timeline:

- **Week 1:** Read papers; download subset; implement preprocessing.
- **Week 2:** Implement baseline CNN; midterm report.
- **Week 3:** Compress/noise robustness experiments.
- **Week 4:** Final evaluation & error analysis; finalize.

---

4. Text Sentiment Analysis with Distilled Transformers

Project Title: Fast Sentiment Analysis Using Distilled Transformers on CPU

Project Aims & Objectives:

1. Fine-tune a distilled transformer (e.g., DistilBERT) for sentiment analysis.
2. Compare with classical methods (LogReg + TF-IDF).
3. Evaluate inference latency and memory usage on CPU.
4. Explore simple model compression (e.g., quantization).

Scope of Work:

- Use small benchmark datasets on local machines.
- Out of scope: pretraining from scratch.

Expected Deliverables:

1. Fine-tuned model and baseline.
2. Performance vs. efficiency analysis.
3. Final report and slides.

Prerequisite Knowledge / Skills:

- Python, HuggingFace Transformers
- Basic NLP (tokenization, embeddings)

Role of Each Student:

- **Student A:** Classical baseline and data preprocessing.
- **Student B:** DistilBERT fine-tuning and experiments.
- **Student C:** Efficiency measurements and report.

Paper(s) Link:

- Sanh et al., “DistilBERT, a Distilled Version of BERT,” 2019. https://arxiv.org/abs/1910.01108
- Devlin et al., “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” 2019. https://arxiv.org/abs/1810.04805

Data Link:

- IMDb or SST-2:
  IMDb: https://ai.stanford.edu/~amaas/data/sentiment/
  SST-2 via HuggingFace: https://huggingface.co/datasets/glue

Code Link (reference):

- HuggingFace sentiment example: https://github.com/huggingface/transformers/tree/main/examples/pytorch/text-classification

Timeline:

- **Week 1:** Learn DistilBERT; prepare dataset.
- **Week 2:** Implement classical baseline; start fine-tuning; midterm report.
- **Week 3:** Complete fine-tuning; run compression/quantization tests.
- **Week 4:** Evaluation, error analysis; final documentation.

---

5. Adversarial Attacks and Defenses on Image Classifiers

Project Title: Exploring Adversarial Robustness of Lightweight Image Classifiers

Project Aims & Objectives:

1. Implement simple adversarial attacks (FGSM, PGD) on a small CNN.
2. Apply basic adversarial training as a defense.
3. Compare clean vs. robust accuracy.
4. Analyze trade-offs in robustness vs. performance.

Scope of Work:

- Use small subset of CIFAR-10 or MNIST.
- Out of scope: complex certified defenses.

Expected Deliverables:

1. Attack & defense implementations.
2. Robustness evaluation plots.
3. Final report & repo.

Prerequisite Knowledge / Skills:

- CNNs, gradient computation
- Basic security concepts (adversarial examples)

Role of Each Student:

- **Student A:** Baseline CNN and training.
- **Student B:** Implement attacks (FGSM, PGD).
- **Student C:** Implement adversarial training and analysis.

Paper(s) Link:

- Madry et al., “Towards Deep Learning Models Resistant to Adversarial Attacks,” ICLR 2018. https://arxiv.org/abs/1706.06083
- Kurakin et al., “Adversarial Machine Learning at Scale,” ICLR 2017. https://arxiv.org/abs/1611.01236

Data Link:

- CIFAR-10: https://www.cs.toronto.edu/~kriz/cifar.html

Code Link (reference):

- MadryLab adversarial examples: https://github.com/MadryLab/cifar10_challenge

Timeline:

- **Week 1:** Baseline CNN; train on small dataset.
- **Week 2:** Implement FGSM/PGD; midterm report.
- **Week 3:** Adversarial training; track robust accuracy.
- **Week 4:** Summarize trade-offs; finalize.

---

6. Sleep Stage Classification from EEG with Lightweight Models

Project Title: EEG-Based Sleep Stage Classification Using Compact Deep Networks

Project Aims & Objectives:

1. Implement a CNN/LSTM model for sleep stage classification.
2. Work with single-channel or few-channel EEG for simplicity.
3. Compare performance to basic feature-based methods.
4. Provide interpretability via channel/temporal attention or saliency.

Scope of Work:

- Use open EEG sleep data (Sleep-EDF).
- Out of scope: multi-night personalized models.

Expected Deliverables:

1. Preprocessing pipeline.

2. Trained deep model and baseline.
3. Interpretability and analysis.

Prerequisite Knowledge / Skills:

- Basic DSP (EEG filtering, epoching)
- Python, 1D CNNs/RNNs

Role of Each Student:

- **Student A:** Preprocessing (filtering, epoching, labeling).
- **Student B:** Deep model implementation and training.
- **Student C:** Baseline, interpretability, and reporting.

Paper(s) Link:

- Supratak et al., "DeepSleepNet: A Model for Automatic Sleep Stage Scoring Based on Raw Single-Channel EEG," IEEE TNSRE, 2017 (method ref). https://doi.org/10.1109/TNSRE.2016.2640960
- Chambon et al., "A Deep Learning Architecture for Temporal Sleep Stage Classification Using Multivariate and Multimodal Time Series," IEEE TBME, 2018. https://doi.org/10.1109/TBME.2017.2721960

Data Link:

- Sleep-EDF Expanded (PhysioNet): https://physionet.org/content/sleep-edfx/1.0.0/

Code Link (reference):

- DeepSleepNet TensorFlow: https://github.com/akaraspt/deepsleepnet

Timeline:

- **Week 1:** Study papers; set up preprocessing.
- **Week 2:** Implement baseline and deep model; midterm report.
- **Week 3:** Train and evaluate; interpretability analyses.
- **Week 4:** Final results, writing, presentation.

---

7. Fake News Detection with Lightweight Transformers

Project Title: Fake News Detection via Distilled Language Models and Explainability

Project Aims & Objectives:

1. Fine-tune a small transformer (e.g., DistilBERT or MiniLM) for fake vs. real news classification.
2. Compare with classical n-gram-based model.
3. Use simple explainability (e.g., LIME/SHAP) for predictions.
4. Analyze failure cases and potential biases.

Scope of Work:

- Use publicly available fake news datasets.
- Out of scope: web crawling.

Expected Deliverables:

1. Fine-tuned model & baseline classifier.
2. Explainability reports and visualizations.
3. Final report & GitHub.

Prerequisite Knowledge / Skills:

- NLP basics, Transformer fine-tuning
- Ethics in data usage

Role of Each Student:

- **Student A:** Data preprocessing and baseline.
- **Student B:** Transformer fine-tuning.
- **Student C:** Explainability and bias/error analysis.

Paper(s) Link:

- Zhou et al., “Fake News Detection via NLP: A Survey,” ACM Comput. Surveys, 2020. https://doi.org/10.1145/3395046
- Wang, “Liar, Liar Pants on Fire: A New Benchmark Dataset for Fake News Detection,” ACL 2017 (dataset ref). https://arxiv.org/abs/1705.00648

Data Link:

- LIAR dataset: https://www.cs.ucsb.edu/~william/software.html
- FakeNewsNet: https://github.com/KaiDMML/FakeNewsNet

Code Link (reference):

- HuggingFace text classification examples: https://github.com/huggingface/transformers/tree/main/examples/pytorch/text-classification

Timeline:

- **Week 1:** Read survey; choose dataset; preprocess.
- **Week 2:** Baseline model; transformer fine-tuning; midterm report.
- **Week 3:** Explainability and error analysis.
- **Week 4:** Finalization.

---

8. Malware Classification from Opcode Sequences

Project Title: Deep Learning for Static Malware Classification Using Opcode Sequences

Project Aims & Objectives:

1. Implement an RNN or 1D CNN model for malware vs. benign classification.
2. Extract opcode sequences or byte-level representations.
3. Compare to classical ML baseline (e.g., n-gram + SVM).
4. Evaluate robustness to obfuscation-like transformations (e.g., noise in sequences).

Scope of Work:

- Use existing dataset with extracted features if binary analysis is too heavy.
- Out of scope: advanced obfuscation detection.

Expected Deliverables:

1. Deep model with training scripts.
2. Baseline and robustness tests.
3. Final report.

Prerequisite Knowledge / Skills:

- Basic cybersecurity concepts
- Python, RNN/CNN for sequences

Role of Each Student:

- **Student A:** Data exploration, feature extraction.
- **Student B:** Deep model implementation.
- **Student C:** Baseline, robustness experiments, writing.

Paper(s) Link:

- Raff et al., “Malware Detection by Eating a Whole EXE,” AAAI 2018 (concept). https://arxiv.org/abs/1710.09435
- Saxe & Berlin, “Deep Neural Network Based Malware Detection Using Two Dimensional Binary Program Features,” 2015 (background). https://arxiv.org/abs/1508.03096

Data Link:

- EMBER dataset: https://github.com/elastic/ember

Code Link (reference):

- EMBER baseline: https://github.com/elastic/ember

Timeline:

- **Week 1:** Understanding dataset; choose representation.
- **Week 2:** Implement baseline; start deep model; midterm report.
- **Week 3:** Training & evaluation; robustness checks.
- **Week 4:** Final documentation.

---

9. Phishing URL Detection with Deep Learning

Project Title: Neural Models for Phishing URL and Website Detection

Project Aims & Objectives:

1. Implement a character-level CNN or RNN for URL classification.
2. Compare with classical features (e.g., length, special chars) + ML.
3. Evaluate on balanced and imbalanced splits.
4. Provide a lightweight model suitable for browser/plugin use.

Scope of Work:

- Use public phishing URL datasets.
- Out of scope: dynamic analysis of page content.

Expected Deliverables:

1. Deep model + baselines.
2. Analysis of false positives/negatives.
3. Final report & repo.

Prerequisite Knowledge / Skills:

- Basic cybersecurity, URL structure
- Python, text modeling

Role of Each Student:

- **Student A:** Data collection and cleaning.
- **Student B:** Deep model implementation.
- **Student C:** Baseline features, imbalanced handling, reporting.

Paper(s) Link:

- Le et al., “URLNet: Learning a URL Representation with Deep Learning for Malicious URL Detection,” NDSS 2018. https://arxiv.org/abs/1802.03162
- Adebowale et al., “Phishing Websites Detection: A Machine Learning Approach,” IEEE Access, 2019. https://doi.org/10.1109/ACCESS.2019.2901982

Data Link:

- Phishtank / Kaggle phishing URL datasets: https://www.phishtank.com/ https://www.kaggle.com/datasets/shashwatwork/phishing-dataset-for-machine-learning

Code Link (reference):

- URLNet repo: https://github.com/Antimalweb/URLNet

Timeline:

- **Week 1:** Select dataset; exploratory analysis.
- **Week 2:** Baseline & deep model; midterm report.

- **Week 3:** Training, imbalance techniques (e.g., weighting).
- **Week 4:** Final evaluation and documentation.

---

10. Speech Emotion Recognition with 1D CNNs

Project Title: Lightweight Speech Emotion Recognition on Open Datasets

Project Aims & Objectives:

1. Implement a CNN-based model for emotion classification using spectrograms.
2. Compare with classical MFCC + SVM baseline.
3. Evaluate cross-speaker generalization.
4. Optimize for CPU inference.

Scope of Work:

- Use small speech emotion dataset (RAVDESS or CREMA-D).
- Out of scope: real-time streaming application.

Expected Deliverables:

1. Preprocessing pipeline (spectrograms).
2. Deep and baseline models.
3. Performance analysis.

Prerequisite Knowledge / Skills:

- Basic audio processing (sampling, STFT)
- CNNs

Role of Each Student:

- **Student A:** Audio preprocessing and baseline.
- **Student B:** CNN implementation & training.
- **Student C:** Evaluation and report.

Paper(s) Link:

- Neumann & Vu, “Attentive Convolutional Neural Network based Speech Emotion Recognition: A Study on the Impact of Input Features, Signal Length, and Acted Speech,” Interspeech 2017. https://arxiv.org/abs/1706.00612
- Tripathi et al., “Deep Neural Networks for Emotion Recognition in Speech,” IJCNN 2019. https://doi.org/10.1109/IJCNN.2019.8852153

Data Link:

- RAVDESS: https://zenodo.org/record/1188976

Code Link (reference):

- Example SER repo: https://github.com/marcogdepinto/emotion-recognition-english

Timeline:

- **Week 1:** Understand data; preprocess.
- **Week 2:** Baseline; CNN; midterm report.
- **Week 3:** Training; cross-speaker experiments.
- **Week 4:** Finalization.

---

11. Multimodal Fake News: Image–Text Inconsistency

Project Title: Detecting Inconsistency Between Image and Text in News Articles

Project Aims & Objectives:

1. Implement a simple multimodal model combining image CNN features and text embeddings.
2. Classify whether the image is semantically consistent with the headline/text.
3. Compare multimodal vs. text-only performance.
4. Perform qualitative analysis of failure cases.

Scope of Work:

- Use available fake news multimodal dataset.
- Out of scope: heavy pretraining.

Expected Deliverables:

1. Data pipeline (image + text).
2. Multimodal model and text-only baseline.
3. Final report.

Prerequisite Knowledge / Skills:

- CNNs for images, transformers/embeddings for text
- PyTorch/TensorFlow

Role of Each Student:

- **Student A:** Image feature extraction.
- **Student B:** Text pipeline and embeddings.
- **Student C:** Multimodal fusion model + analysis.

Paper(s) Link:

- Jin et al., “Multimodal Fusion with Co-Attention Networks for Fake News Detection,” ACL 2017. https://aclanthology.org/D17-1317/
- Khattar et al., “MVA: Multimodal Variational Autoencoder for Fake News Detection,” WWW 2019. https://arxiv.org/abs/1901.06794

Data Link:

- Weibo or Twitter multimodal fake news datasets (e.g., FakeNewsNet multimodal subset): https://github.com/KaiDMML/FakeNewsNet

Code Link (reference):

- FakeNewsNet baseline: https://github.com/KaiDMML/FakeNewsNet/tree/master/code

Timeline:

- **Week 1:** Set up dataset; prepare image and text inputs.
- **Week 2:** Implement text-only model; midterm report.
- **Week 3:** Implement and train multimodal model.
- **Week 4:** Analysis and documentation.

---

12. Medical Image Segmentation with UNet Variants

Project Title: UNet-Based Segmentation for Small Medical Datasets

Project Aims & Objectives:

1. Implement a small UNet for binary segmentation (e.g., nuclei, lesions).
2. Compare against a simpler baseline (e.g., thresholding).
3. Perform data augmentation to mitigate small dataset size.
4. Evaluate with Dice and IoU metrics.

Scope of Work:

- Use small medical segmentation dataset (e.g., Kaggle 2018 Data Science Bowl, DRIVE).
- Out of scope: 3D volumetric segmentation.

Expected Deliverables:

1. Segmentation model code and trained weights.
2. Evaluation metrics and visual results.
3. Final report.

Prerequisite Knowledge / Skills:

- CNNs, segmentation concepts
- Image preprocessing

Role of Each Student:

- **Student A:** Data preprocessing & augmentation.
- **Student B:** UNet implementation and training.
- **Student C:** Evaluation, baselines, and report.

Paper(s) Link:

- Ronneberger et al., “U-Net: Convolutional Networks for Biomedical Image Segmentation,” MICCAI 2015. https://arxiv.org/abs/1505.04597
- Zhou et al., “UNet++: A Nested U-Net Architecture for Medical Image Segmentation,” DLMIA 2018. https://arxiv.org/abs/1807.10165

Data Link:

- 2018 Data Science Bowl (nuclei): https://www.kaggle.com/competitions/data-science-bowl-2018/data

Code Link (reference):

- UNet PyTorch implementations: https://github.com/milesial/Pytorch-UNet

Timeline:

- **Week 1:** Explore dataset; set up preprocessing.
- **Week 2:** Implement UNet; baseline thresholding; midterm report.
- **Week 3:** Train and tune; augmentations.
- **Week 4:** Final evaluation and write-up.

---

13. Time-Series Anomaly Detection in Network Traffic

Project Title: LSTM-Based Anomaly Detection in Network Flow Time Series

Project Aims & Objectives:

1. Implement an LSTM/GRU-based model for anomaly detection (e.g., autoencoder or prediction-based).
2. Use public network traffic data (e.g., CIC-IDS).
3. Compare to unsupervised classical methods (Isolation Forest, LOF).
4. Visualize detected anomalies and investigate patterns.

Scope of Work:

- Focus on a few key features (flows, bytes, ports).
- Out of scope: high-speed deployment systems.

Expected Deliverables:

1. Preprocessed time-series dataset.
2. LSTM model and classical baselines.
3. Anomaly analysis.

Prerequisite Knowledge / Skills:

- Basic networking concepts
- RNNs / time-series modeling

Role of Each Student:

- **Student A:** Data selection and feature engineering.
- **Student B:** Deep model implementation.
- **Student C:** Baseline classical models and analysis.

Paper(s) Link:

- Mirsky et al., “Kitsune: An Ensemble of Autoencoders for Online Network Intrusion Detection,” NDSS 2018. https://arxiv.org/abs/1802.09089

- Vinayakumar et al., “Deep Learning for Network Anomaly Detection: A Survey,” IEEE Comm. Surveys, 2019. https://doi.org/10.1109/COMST.2019.2899779

Data Link:

- CIC-IDS2017: https://www.unb.ca/cic/datasets/ids-2017.html

Code Link (reference):

- Example LSTM anomaly detection: https://github.com/rob-med/awesome-TS-anomaly-detection

Timeline:

- **Week 1:** Understand dataset; build simple time-series features.
- **Week 2:** Implement LSTM and baselines; midterm report.
- **Week 3:** Train, tune, detect anomalies.
- **Week 4:** Summarize and present.

---

14. Question Answering with Lightweight Models

Project Title: Efficient Extractive Question Answering on SQuAD with Distilled Models

Project Aims & Objectives:

1. Fine-tune DistilBERT or TinyBERT on a subset of SQuAD.
2. Compare performance vs. size/latency trade-offs.
3. Evaluate robustness to paraphrased questions.
4. Explore small modifications (e.g., layer freezing, reduced max sequence length).

Scope of Work:

- Use SQuAD v1.1 subset due to computational limits.
- Out of scope: training from scratch.

Expected Deliverables:

1. Fine-tuned QA model.
2. Performance and efficiency analysis.
3. Final report & slides.

Prerequisite Knowledge / Skills:

- Transformers, tokenization
- Python, HuggingFace

Role of Each Student:

- **Student A:** Data subset and baseline config.
- **Student B:** Fine-tuning experiments.
- **Student C:** Robustness tests and analysis.

Paper(s) Link:

- Sanh et al., DistilBERT (as above).
- Jiao et al., “TinyBERT: Distilling BERT for Natural Language Understanding,” ACL 2020. https://arxiv.org/abs/1909.10351

Data Link:

- SQuAD v1.1: https://rajpurkar.github.io/SQuAD-explorer/

Code Link (reference):

- HuggingFace QA examples: https://github.com/huggingface/transformers/tree/main/examples/pytorch/question-answering

Timeline:

- **Week 1:** Understand QA task; set up small subset.
- **Week 2:** Baseline fine-tuning; midterm report.

- **Week 3:** Efficiency + robustness experiments.
- **Week 4:** Finalization.

---

15. Document Image Classification (Receipts, Forms)

Project Title: Document Image Classification using Lightweight CNNs

Project Aims & Objectives:

1. Train a small CNN to classify document types (e.g., invoice, receipt, form).
2. Compare raw image vs. text-only (OCR) approaches.
3. Evaluate performance on rotated and noisy documents.
4. Propose a simple ensemble or fusion strategy.

Scope of Work:

- Use public document image dataset.
- Out of scope: full OCR system development (use existing OCR like Tesseract).

Expected Deliverables:

1. CNN model and text-based baseline.
2. Robustness evaluation (rotation, noise).
3. Final report.

Prerequisite Knowledge / Skills:

- CNNs, basic OCR usage
- Image augmentation

Role of Each Student:

- **Student A:** Data collection and augmentation.
- **Student B:** CNN implementation and training.
- **Student C:** OCR-based baseline and fusion, reporting.

Paper(s) Link:

- Harley et al., “Evaluation of Deep Convolutional Nets for Document Image Classification and Retrieval,” ICDAR 2015. https://arxiv.org/abs/1502.07058

Data Link:

- RVL-CDIP dataset (if accessible) or smaller open datasets; e.g.: https://www.cs.cmu.edu/~aharley/rvl-cdip/

Code Link (reference):

- Simple document classification repo: https://github.com/harveyslash/Document-Image-Classification

Timeline:

- **Week 1:** Obtain dataset; basic pipeline.
- **Week 2:** CNN baseline; OCR baseline; midterm report.
- **Week 3:** Fusion and robustness tests.
- **Week 4:** Finalization.

---

16. Tabular Data Fraud Detection with Deep & Classical Models

Project Title: Credit Card Fraud Detection: Comparing Deep Learning with Classical Methods

Project Aims & Objectives:

1. Implement an MLP model for fraud detection on tabular data.
2. Compare with classical methods (LogReg, Random Forest, XGBoost).
3. Address severe class imbalance with techniques (SMOTE, weighting).
4. Evaluate using ROC-AUC, PR-AUC, and cost-based metrics.

Scope of Work:

- Use public credit card fraud dataset.

- Out of scope: real-time deployment.

Expected Deliverables:

1. Preprocessing and ML pipeline.
2. Deep + classical model performance comparison.
3. Final report.

Prerequisite Knowledge / Skills:

- scikit-learn, MLP basics
- Imbalanced data techniques

Role of Each Student:

- **Student A:** Data exploration & preprocessing.
- **Student B:** Classical models.
- **Student C:** Deep model & reporting.

Paper(s) Link:

- Carcillo et al., “Scarff: A Scalable Framework for Streaming Credit Card Fraud Detection with Spark,” IEEE Big Data 2018 (context). https://doi.org/10.1109/BigData.2018.8622522
- Dal Pozzolo et al., “Calibrating Probability with Undersampling for Unbalanced Classification,” IEEE Symposium on Computational Intelligence, 2015. https://arxiv.org/abs/1505.05497

Data Link:

- Kaggle Credit Card Fraud Detection: https://www.kaggle.com/mlg-ulb/creditcardfraud

Code Link (reference):

- Example fraud detection notebooks on Kaggle (for ideas, not copying).

Timeline:

- **Week 1:** Data prep and baseline.
- **Week 2:** Classical models; midterm report.
- **Week 3:** Deep model, imbalance handling.
- **Week 4:** Final comparison and report.

---

17. Low-Resource Machine Translation with Tiny Transformers

Project Title: Neural Machine Translation in a Low-Resource Setting with Tiny Models

Project Aims & Objectives:

1. Train or fine-tune a tiny Transformer model for a small parallel corpus (e.g., Eng–German subset or EN–FR).
2. Compare to a phrase-based baseline (if feasible) or naive dictionary-based translation.
3. Evaluate BLEU and qualitative quality.
4. Explore basic regularization for low-resource.

Scope of Work:

- Use ~50k sentence pairs; CPU training.
- Out of scope: large-scale training.

Expected Deliverables:

1. MT model and baseline.
2. Evaluation (BLEU, examples).
3. Final report.

Prerequisite Knowledge / Skills:

- Basic MT concepts
- Transformer architectures

Role of Each Student:

- **Student A:** Data preprocessing (tokenization, BPE).
- **Student B:** Model training.
- **Student C:** Evaluation and analysis.

Paper(s) Link:

- Vaswani et al., “Attention Is All You Need,” NeurIPS 2017. https://arxiv.org/abs/1706.03762
- Hieber et al., “Sockeye 2: A Toolkit for Neural Machine Translation,” 2020. https://arxiv.org/abs/2005.07129

Data Link:

- WMT14 EN–DE (small subset) or TED talks parallel corpora (e.g., IWSLT): https://wit3.fbk.eu/ or https://www.statmt.org/wmt14/translation-task.html

Code Link (reference):

- Fairseq or Marian examples: https://github.com/facebookresearch/fairseq

Timeline:

- **Week 1:** Dataset subset; tokenization.
- **Week 2:** Baseline; tiny Transformer; midterm report.
- **Week 3:** Training & evaluation.
- **Week 4:** Finalization.

---

18. Style Transfer with Efficient Neural Nets

Project Title: Real-Time Neural Style Transfer on CPU

Project Aims & Objectives:

1. Implement a feed-forward style transfer network (Johnson architecture).
2. Compare runtime and quality vs. iterative optimization (Gatys).
3. Evaluate on multiple content and style images.
4. Explore small tweaks (e.g., fewer channels for efficiency).

Scope of Work:

- Focus on single style or a few styles.
- Out of scope: multi-style or arbitrary style generalization.

Expected Deliverables:

1. Style transfer model and scripts.
2. Visual comparisons and benchmarks.
3. Final report.

Prerequisite Knowledge / Skills:

- CNNs, content & style loss concepts
- PyTorch/TensorFlow

Role of Each Student:

- **Student A:** Implement Gatys-style optimization.
- **Student B:** Feed-forward style transfer network.
- **Student C:** Evaluation and analysis.

Paper(s) Link:

- Gatys et al., “Image Style Transfer Using Convolutional Neural Networks,” CVPR 2016. https://arxiv.org/abs/1508.06576
- Johnson et al., “Perceptual Losses for Real-Time Style Transfer and Super-Resolution,” ECCV 2016. https://arxiv.org/abs/1603.08155

Data Link:

- Use any natural image dataset (e.g., COCO subset) for training; style images from online art (ensure license).

Code Link (reference):

- PyTorch example style transfer: https://pytorch.org/tutorials/advanced/neural_style_tutorial.html

Timeline:

- **Week 1:** Study style transfer; implement optimization-based method.
- **Week 2:** Implement feed-forward model; midterm report.
- **Week 3:** Training; visual evaluation.
- **Week 4:** Finalization.

---

19. Explainable Image Classification (Grad-CAM & Beyond)

Project Title: Interpretable CNNs: Visual Explanations for Image Classification

Project Aims & Objectives:

1. Train a small CNN (or fine-tune ResNet18) on CIFAR-10.
2. Implement Grad-CAM and related explanation methods.
3. Evaluate quality of explanations (sanity checks).
4. Explore effect of model changes on explanations.

Scope of Work:

- Focus on small dataset and small models.
- Out of scope: formal quantitative explanation evaluation.

Expected Deliverables:

1. Trained models.
2. Explanation visualizations.
3. Analysis report.

Prerequisite Knowledge / Skills:

- CNNs, backprop basics
- Visualization in Python

Role of Each Student:

- **Student A:** CNN training.
- **Student B:** Grad-CAM implementation.
- **Student C:** Analysis and reporting.

Paper(s) Link:

- Selvaraju et al., “Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization,” ICCV 2017. https://arxiv.org/abs/1610.02391
- Adebayo et al., “Sanity Checks for Saliency Maps,” NeurIPS 2018. https://arxiv.org/abs/1810.03292

Data Link:

- CIFAR-10: https://www.cs.toronto.edu/~kriz/cifar.html

Code Link (reference):

- Pytorch-Grad-CAM: https://github.com/jacobgil/pytorch-grad-cam

Timeline:

- **Week 1:** Train model on CIFAR-10.
- **Week 2:** Implement Grad-CAM; midterm report.
- **Week 3:** Perform sanity checks; compare variants.

- **Week 4:** Finalization.

---

20. Contrastive Learning for Image Representations

Project Title: Self-Supervised Contrastive Learning on Small Image Datasets

Project Aims & Objectives:

1. Implement a simplified SimCLR-like framework on CIFAR-10.
2. Use learned representations for downstream classification (linear probe).
3. Compare with fully supervised baseline.
4. Analyze the impact of augmentations.

Scope of Work:

- Use small batch sizes and small backbone (e.g., ResNet18).
- Out of scope: large-scale self-supervised pretraining.

Expected Deliverables:

1. Contrastive training script.
2. Linear evaluation and comparisons.
3. Final report.

Prerequisite Knowledge / Skills:

- CNNs, contrastive loss
- Data augmentation

Role of Each Student:

- **Student A:** Implement data augmentation pipeline.
- **Student B:** Contrastive learning framework.
- **Student C:** Linear evaluation and reporting.

Paper(s) Link:

- Chen et al., “A Simple Framework for Contrastive Learning of Visual Representations,” ICML 2020. https://arxiv.org/abs/2002.05709

Data Link:

- CIFAR-10: (above)

Code Link (reference):

- SimCLR in PyTorch: https://github.com/leftthomas/SimCLR

Timeline:

- **Week 1:** Understand SimCLR; basic pipeline.
- **Week 2:** Implement contrastive training; midterm report.
- **Week 3:** Train representations; linear probe.
- **Week 4:** Compare with supervised baseline; finalize.

Midterm Report Template (IEEE-style, ~5-8 pages)

Use IEEE conference format (two-column). Template:
https://www.ieee.org/conferences/publishing/templates.html
Sections:

1. **Title and Abstract (150–200 words)**
   - Problem, dataset, model idea, current status.
2. **Introduction (0.5–1 page)**
   - Problem statement and motivation.
   - Project scope and goals.
3. **Related Work (0.5–1 page)**
   - 2–4 key papers.
   - How your project builds on them / differs.
4. **Methodology (1–1.5 pages)**
   - Data description (source, preprocessing).
   - Model architecture(s) (diagrams encouraged).
   - Baselines you plan to implement.
5. **Preliminary Experiments and Results (~1 page)**
   - Any initial training runs, metrics on small subset.
   - Implementation details (hardware, libraries).
6. **Planned Work and Timeline (0.5 page)**
   - Remaining experiments, ablations.
   - Risks and mitigation strategies.
7. **Team Contributions**
   - Bullet list of each member's work so far and future responsibilities.
8. **References**

Final Report Template (IEEE-style, ~10–15 pages)

Same IEEE format.

1. **Title and Abstract**
   - Summarize problem, method, results, and contributions.
2. **Introduction**
   - Background and real-world relevance.
   - Clear problem definition.
   - Contributions (e.g., "We reproduce X, propose Y extension, and analyze Z").
3. **Related Work**
   - 5–10 key references.
   - Compare methods conceptually.
4. **Data and Preprocessing**
   - Dataset details (size, splits, sources, licenses).
   - Preprocessing steps (filters, tokenization, normalization).
   - Train/validation/test setup.
5. **Methodology**
   - Model architectures (with diagrams if possible).
   - Training procedure (losses, learning rates, optimizers, epochs, batch sizes).

    - Baselines and ablations.
6. **Experiments and Results**
    - Quantitative results (tables, figures).
    - Comparison with baselines and/or reported numbers from papers.
    - Efficiency (runtime, memory) where relevant.
    - Ablation studies (e.g., without augmentation, smaller model).
7. **Discussion**
    - Interpretation of results; strengths and weaknesses.
    - Error analysis and failure cases.
    - Limitations and future work (including what would be needed for publication).
8. **Conclusion**
    - Key takeaways.
    - Potential impact or next steps.
9. **Team Contributions**
    - Clear breakdown per student (design, coding, experiments, writing).
10. **References**
11. **Appendix (optional)**
    - Extra figures, hyperparameter tables, additional results.

Presentation Requirements (20 minutes)

Recommended structure:

1. **Motivation & Problem (3–4 min)**
2. **Data & Method (6–7 min)**
3. **Results & Analysis (6–7 min)**
4. **Limitations & Future Work (2–3 min)**

All team members must speak.

General Grading Rubric (100 points)

1. Technical Depth & Correctness (30 pts)

- Accurate implementation of models and baselines (10)
- Proper experimental protocol (splits, metrics, reproducibility) (10)
- Sound methodology, no major conceptual mistakes (10)

2. Originality & Insight (20 pts)

- Thoughtful extensions or ablations beyond pure reproduction (10)
- Quality of analysis and interpretation of results (10)

3. Implementation & Reproducibility (20 pts)

- Clean, well-organized GitHub repo with documentation (10)
- Can reproduce main results with provided scripts (10)

4. Written Reports (20 pts)

- Midterm report completeness and clarity (5)
- Final report structure, clarity, and depth (10)
- Correct and adequate citation of prior work (5)

5. Presentation & Teamwork (10 pts)

- Clear, well-timed presentation; all members contribute (5)
- Clear description of individual responsibilities and balanced workload (5)

Bonus up to **+5 pts** for publishable-level results (e.g., near state-of-the-art on subset, strong novel analysis, or submission-ready draft).

Policy on GenAI Use

To ensure authentic learning and fair assessment:

Allowed GenAI use (with disclosure):

- Brainstorming ideas and clarifying concepts (e.g., “Explain cross-entropy loss”).
- Assistance with debugging **after** you attempt to fix it.
- Help with English phrasing (grammar, minor style edits).

Not allowed:

- Generating code that you then claim as your own without understanding it.
- Generating large portions of the report (sections, literature review) via GenAI.
- Generating experimental results or “fake” figures/tables.
- Asking GenAI to directly implement the assigned paper or project end-to-end.

Neutralization mechanisms:

1. **Oral Defense & Q&A:**
    - In certain cases, each team member will be asked detailed technical questions (e.g., why you chose a learning rate, implications of certain layers).
    - Inability to explain code or design choices will significantly reduce credit for that part, regardless of how good the code looks.
2. **Code Review:**
    - Instructors may ask you to walk through key functions line-by-line.
    - Random code snippets may be chosen for explanation.
3. **Reproducibility Checks:**
    - The instructor may run your code in a clean environment.
    - If scripts break or hyperparameters are undocumented, points are deducted.
4. **Process Documentation:**
    - You must maintain a brief development log (in `LOG.md`) describing weekly progress, key decisions, and issues encountered.
    - Logs must be consistent with Git commit history.
5. **GenAI Use Disclosure:**
    - Add a short section in both midterm and final reports titled “Use of GenAI Tools” describing where/how you used them (if at all).
    - Honest disclosure will not be penalized; hidden use discovered later will be.
6. **Writing Style Consistency:**
    - Report writing style will be compared with previous coursework when available. Abrupt, unexplained shifts may trigger closer review and questioning.

By combining technical questioning, code walkthroughs, and reproducibility checks, reliance on GenAI for core intellectual work will yield little advantage. Understanding and original thought are what will be graded.